Cours UML & POO Java	Projet de gestion d'un supermarché	Enseignant : Ing. HODABALO Lidaou Année Académique : 2025-2026

PROJET DE GESTION D'UN SUPERMARCHÉ

CONTEXTE GÉNÉRAL DU PROJET :

La société **SUPER MARKET PLUS** est une entreprise spécialisée dans la grande distribution. Elle possède un supermarché qui propose plusieurs familles de produits :

- produits alimentaires ;
- produits frais ;
- produits électroniques ;
- produits d'entretien ;
- produits de boulangerie ;
- produits de boucherie ;
- produits de poissonnerie.

Chaque jour, plusieurs opérations sont réalisées :

- réception des marchandises provenant des fournisseurs ;
- stockage des produits ;
- mise en rayon ;
- vente aux clients ;
- encaissement ;
- édition des factures ;
- suivi financier.

Actuellement, la majorité des opérations sont suivies manuellement avec des fichiers Excel. Cette méthode entraîne plusieurs problèmes :

- difficulté à connaître le stock réel ;
- rupture imprévue de certains produits ;
- erreurs de prix en caisse ;
- mauvais suivi des produits périmés ;
- difficulté à connaître les ventes réalisées ;
- absence de statistiques fiables pour la Direction.

Afin d'améliorer son fonctionnement, la Direction souhaite développer une application informatique complète nommée : **SUPERMARKET MANAGER**
Cette application devra permettre aux différents employés d'effectuer leurs tâches quotidiennes à travers une interface graphique simple.

OBJECTIFS DU PROJET :

L'étudiant devra développer une application Java permettant :

- la gestion des produits ;
- la gestion des fournisseurs ;

- la gestion des commandes ;
- la gestion du stock ;
- la gestion des rayons ;
- la gestion du personnel ;
- la gestion des clients ;
- la vente en caisse ;
- la gestion des paiements ;
- l'impression des factures ;
- l'affichage des statistiques.

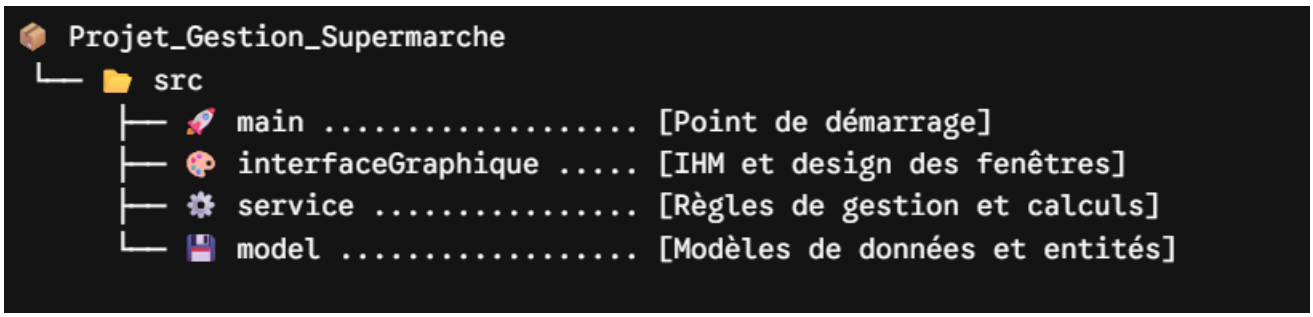
L'application devra respecter :

- les principes de la Programmation Orientée Objet ;
- l'encapsulation ;
- l'héritage ;
- le polymorphisme ;
- les associations entre classes ;
- l'utilisation des collections Java.

Une interface graphique Java Swing devra permettre aux utilisateurs d'utiliser toutes les fonctionnalités.

ARCHITECTURE OBLIGATOIRE DU PROJET :

Le projet devra respecter l'organisation suivante :



Package model

Contient les classes métier :
Produit.java
Client.java
Employe.java
Vente.java
Rayon.java
Fournisseur.java
Paiement.java
Facture.java

Package interfaceGraphique

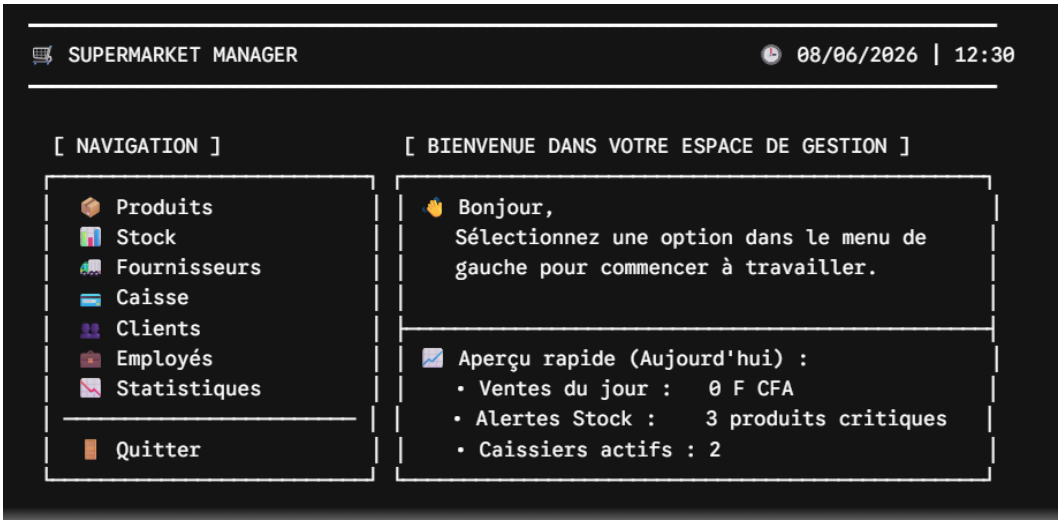
Contient toutes les fenêtres :
Accueil.java
FenetreProduit.java
FenetreStock.java
FenetreCaisse.java
FenetreClient.java
FenetreEmploye.java
Dashboard.java

FENÊTRE PRINCIPALE DE L'APPLICATION

Au lancement, l'utilisateur doit accéder à un menu général.
Créer :
Accueil.java
Cette fenêtre représente l'entrée de l'application.
Elle devra contenir :

- titre de l'application ;
- boutons d'accès aux modules ;
- menu principal.

Exemple :



Chaque bouton devra ouvrir la fenêtre correspondante.

EXERCICE 1 : GESTION DES PRODUITS

Description métier
Les produits représentent l'élément principal du supermarché.
Avant d'être vendus, ils doivent être enregistrés dans l'application.
Chaque produit doit être identifié afin de connaître :

- son prix ;
- son stock disponible ;
- son rayon ;
- sa catégorie.

Le gestionnaire doit pouvoir ajouter de nouveaux produits lorsque le supermarché commercialise de nouveaux articles.

Créer la classe :
Produit
Attributs :
reference : int
designation : String
prixAchat : double
prixVente : double
quantiteStock : int

Méthodes :
ajouterStock()
retirerStock()
modifierPrix()
calculerMarge()
toString()

Gestion des catégories
Créer les classes :
Produit
▲
ProduitFrais
ProduitElectronique
ProduitArtisanal

ProduitFrais possède :
datePeremption
temperatureConservation

ProduitElectronique possède :
marque
garantie

ProduitArtisanal concerne :
- *boulangerie ;*
- *poissonnerie ;*
- *boucherie.*

Interface graphique demandée
Créer :
FenetreProduit.java
Cette interface doit permettre au gestionnaire :
- d'ajouter un produit ;
- modifier un produit ;
- supprimer un produit ;
- rechercher un produit.

L'écran devra contenir :



EXERCICE 2 : GESTION FOURNISSEURS ET APPROVISIONNEMENT
Description métier
Un supermarché travaille avec plusieurs fournisseurs.
Lorsqu'un produit manque :

- une commande est créée ;
- le fournisseur livre ;
- le magasinier réceptionne ;
- le stock augmente.

Créer :
Fournisseur
Attributs :
code
nom
telephone
adresse

Méthodes :
ajouterProduit()
afficher()

Créer :
CommandeFournisseur
Elle contient :
numeroCommande
date
fournisseur
listeProduits
etatCommande

Interface Fournisseur
Créer :
FenetreFournisseur.java
Elle doit permettre :
Nouveau fournisseur
Modifier fournisseur
Liste fournisseurs
Nouvelle commande

EXERCICE 3 : GESTION DU STOCK
Description métier
Le stock doit être automatiquement mis à jour.
Trois mouvements existent :

- entrée fournisseur ;
- sortie vente ;
- retrait produit périmé.

Créer les traitements permettant :
entreeStock()
sortieStock()
verifierStockMinimum()

Interface Stock
Créer :
FenetreStock.java
Elle doit afficher :
Produit
Quantité disponible
Seuil minimum
Etat stock
Avec alertes :
STOCK FAIBLE
PRODUIT EXPIRE

EXERCICE 4 : GESTION DES RAYONS DU SUPERMARCHÉ

Description métier

Dans un supermarché, les produits ne sont pas simplement stockés. Ils doivent être organisés dans différents espaces appelés **rayons** afin de faciliter leur vente.
Un rayon correspond à une zone commerciale regroupant des produits de même catégorie.
Exemple :

- Rayon Alimentaire ;
- Rayon Boissons ;
- Rayon Électronique ;
- Rayon Boulangerie ;
- Rayon Boucherie ;
- Rayon Hygiène.

Chaque rayon possède un responsable appelé **Chef de rayon**.
Le chef de rayon est chargé :

- de suivre les produits disponibles ;
- de vérifier la présentation des articles ;
- de demander un réapprovisionnement ;
- de retirer les produits non conformes ;
- de suivre la performance commerciale du rayon.

L'application devra permettre de gérer ces différentes opérations.

Créer la classe :
Rayon
Attributs :

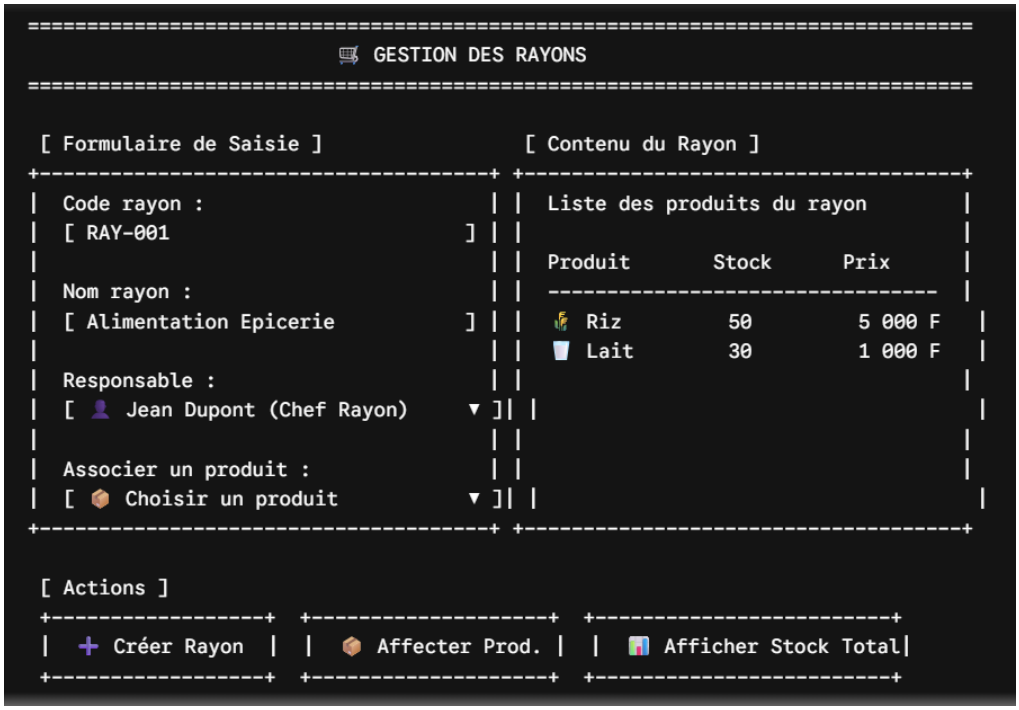
private int codeRayon;
private String nomRayon;
private ChefRayon responsable;
private ArrayList<Produit> listeProduits;

Méthodes :
ajouterProduit()
retirerProduit()
rechercherProduit()
afficherProduits()
calculerValeurStock()

La méthode *calculerValeurStock()* devra retourner la valeur totale des marchandises présentes dans le rayon.
Formule : *Somme (prixVente × quantitéDisponible)*

Interface graphique demandée
Créer :
FenetreRayon.java
Cette interface sera utilisée par les chefs de rayon.
Elle devra permettre :

- créer un rayon ;
- affecter un produit à un rayon ;
- changer le responsable ;
- consulter les produits.
- Présentation attendue :



EXERCICE 5 : GESTION DU PERSONNEL DU SUPERMARCHÉ
Description métier
Le fonctionnement du supermarché repose sur plusieurs catégories d'employés.
Chaque employé intervient dans un processus précis.
La Direction souhaite suivre :

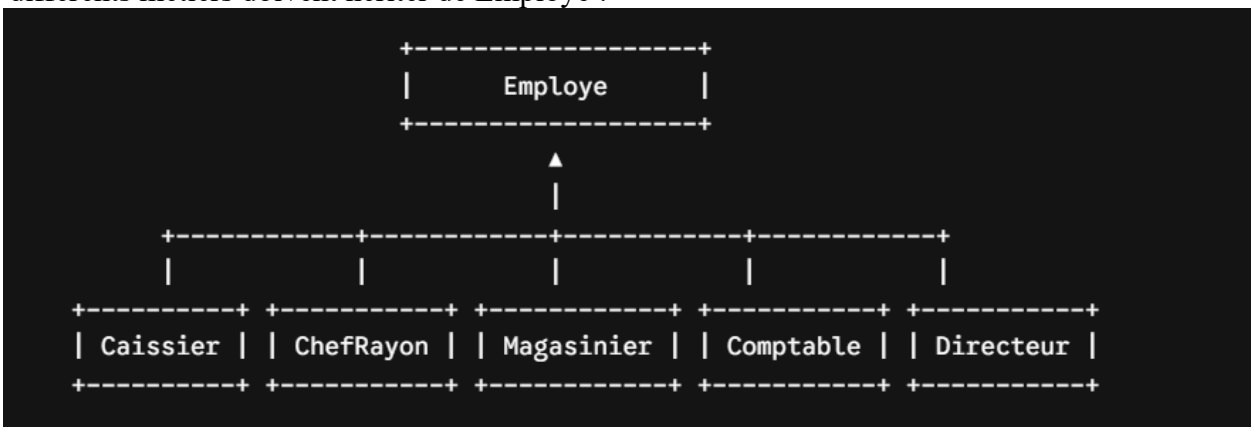
- l'identité des employés ;
- leurs fonctions ;
- leurs responsabilités ;
- leurs salaires.

Tous les employés partagent des informations communes.
Créer une classe générale :

Employe
Attributs :
private int matricule;
private String nom;
private String prenom;
private double salaire;

Méthodes :
afficherInformations()
calculerPrime()
afficherRole()

Les différents métiers doivent hériter de Employe :



Classe Caissier
Un caissier est chargé de gérer les ventes.
Ajouter :
numeroCaisse

Méthodes :
ouvrirCaisse()
effectuerVente()
encaisserPaiement()

Classe Magasinier
Le magasinier gère les entrées de marchandises.
Méthodes :
receptionnerLivraison()
mettreAJourStock()

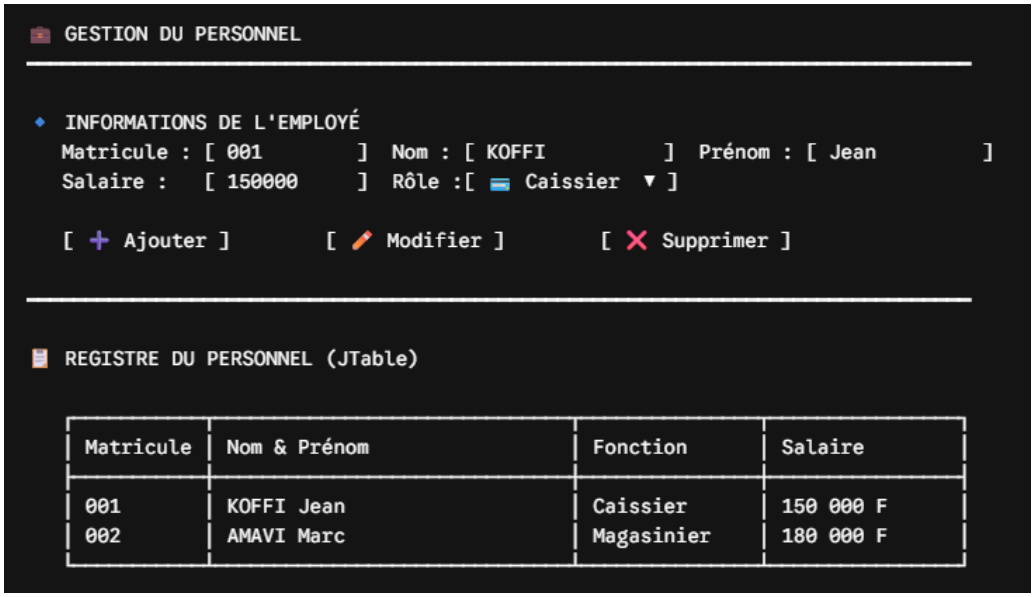
Classe Comptable
Le comptable assure le suivi financier.
Méthodes :
calculerRecette()
genererRapport()

Classe Directeur
Le directeur supervise toute l'activité.
Méthodes :
voirStatistiques()
consulterEmployes()

Interface graphique demandée
Créer :
FenetreEmploye.java
Elle devra permettre :

- enregistrer un employé ;
- modifier ses informations ;
- rechercher un employé ;
- afficher la liste du personnel.

Exemple :



EXERCICE 6 : GESTION DES CLIENTS ET FIDÉLITÉ
Description métier
Le supermarché souhaite fidéliser ses clients réguliers.
Chaque client peut disposer d'une carte fidélité.
Lorsqu'un client réalise des achats :

- la vente est enregistrée ;
- son historique est conservé ;
- des points fidélité sont calculés.

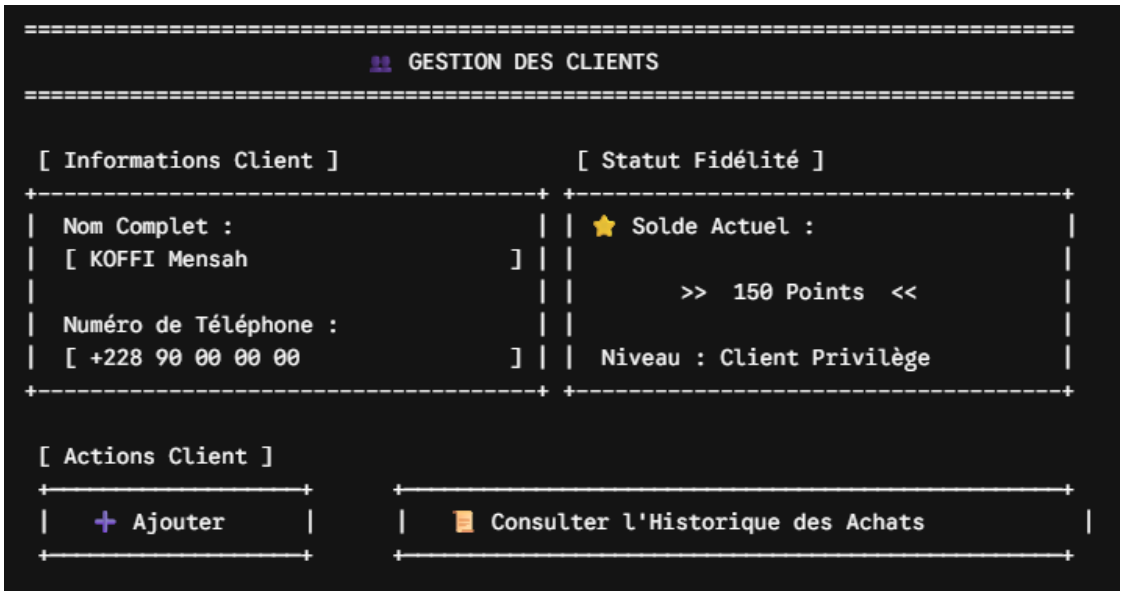
Ces points pourront plus tard permettre d'obtenir des avantages.

Créer :
Client
Attributs :
private int numeroClient;
private String nom;
private String telephone;
private int pointsFidelite;
private ArrayList<Vente> historiqueAchats;

Méthodes :
ajouterPoints()
consulterHistorique()
afficherClient()

Interface graphique demandée
Créer :
FenetreClient.java
Fonctionnalités :
Nouveau client
Modifier client
Recherche client
Historique achats
Points fidélité

Présentation :



EXERCICE 7 : MODULE CAISSE ET VENTE
Description métier
Cette partie représente le cœur du supermarché.
Lorsqu'un client termine ses achats, il passe en caisse.
Le processus réel est :

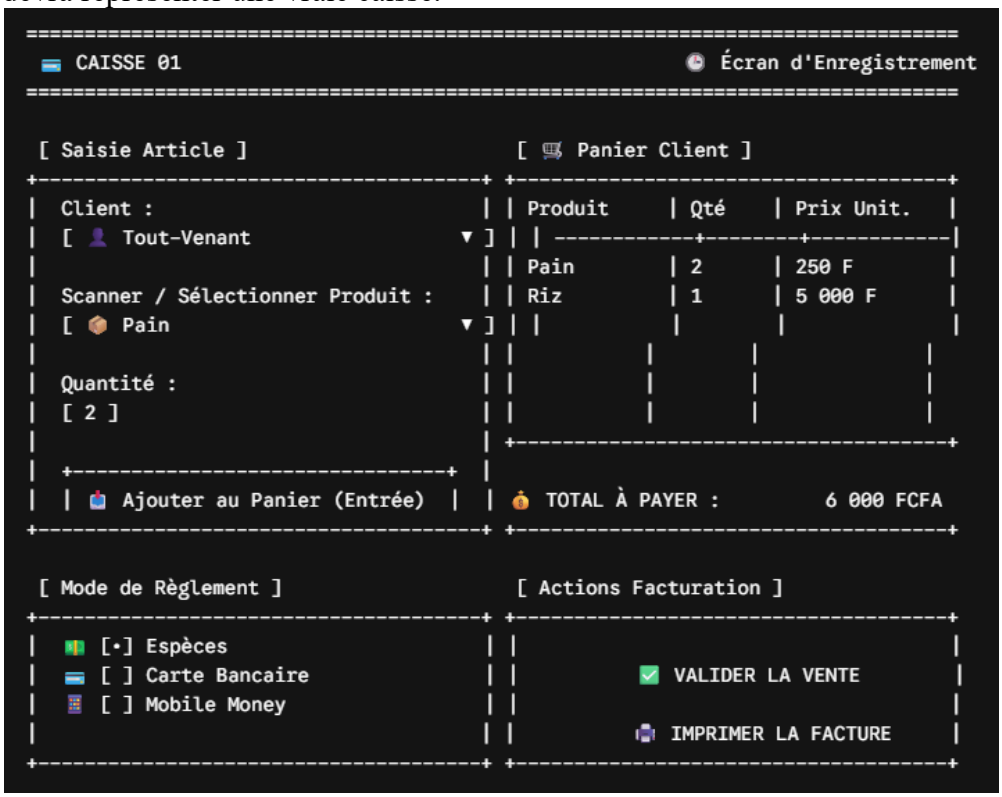
1. Le caissier ouvre une nouvelle vente.

2. Les produits sont scannés.
3. Le système vérifie les stocks.
4. Les articles sont ajoutés au panier.
5. Le montant total est calculé.
6. Le client effectue son paiement.
7. Une facture est générée.
8. Le stock diminue automatiquement.

Créer :
Vente
Attributs :
private int numeroVente;
private Date date;
private Client client;
private Caissier caissier;
private ArrayList<Produit> panier;

Méthodes :
ajouterProduit()
supprimerProduit()
calculerTotal()
validerVente()

Interface Caisse
Créer :
FenetreCaisse.java
Cette fenêtre devra représenter une vraie caisse.



Après validation :
Le programme doit :

- enregistrer la vente ;
- réduire le stock ;
- enregistrer paiement ;

Créer la classe :
Paiement
Attributs :
private int numeroPaiement;
private double montant;
private String modePaiement;
private Date datePaiement;
private Vente vente;

Méthodes :
validerPaiement()
afficherPaiement()

Interface graphique Paiement
Dans la fenêtre de caisse, le paiement doit être intégré.
L'utilisateur doit pouvoir sélectionner :
Mode de paiement :
( ) Espèces
( ) Carte bancaire
( ) Mobile Money

Lorsque le caissier clique sur :
[VALIDER PAIEMENT]
le système doit :

- *vérifier le montant ;*
- *enregistrer le paiement ;*
- *confirmer la vente ;*
- *afficher un message de succès.*

Exemple de message :
Paiement validé avec succès.
Facture prête à être imprimée.

EXERCICE 9 : GESTION DES FACTURES
Description métier
Après chaque vente validée, une facture doit être générée automatiquement.
La facture constitue la preuve d'achat remise au client.
Elle doit reprendre :

- le numéro de facture ;
- la date ;
- le nom du client s'il est connu ;
- le nom du caissier ;
- la liste des produits achetés ;

- les quantités ;
- les prix unitaires ;
- le montant total ;
- le mode de paiement

Créer la classe :
Facture
Attributs :
private int numeroFacture;
private Date dateFacture;
private Vente vente;
private Paiement paiement;

Méthodes :
genererFacture()
imprimerFacture()
afficherFacture()

Interface graphique Facture
Créer :
FenetreFacture.java
Cette fenêtre doit afficher le reçu du client.
Exemple :



La fenêtre devra contenir au minimum :
[IMPRIMER]
[FERMER]

L'impression réelle sur papier n'est pas obligatoire, mais l'affichage clair de la facture est exigé.

EXERCICE 10 : TABLEAU DE BORD DIRECTION

Description métier

Le directeur du supermarché doit disposer d'un tableau de bord pour suivre l'activité globale.
Ce tableau de bord doit l'aider à prendre des décisions.
Il doit permettre de connaître :

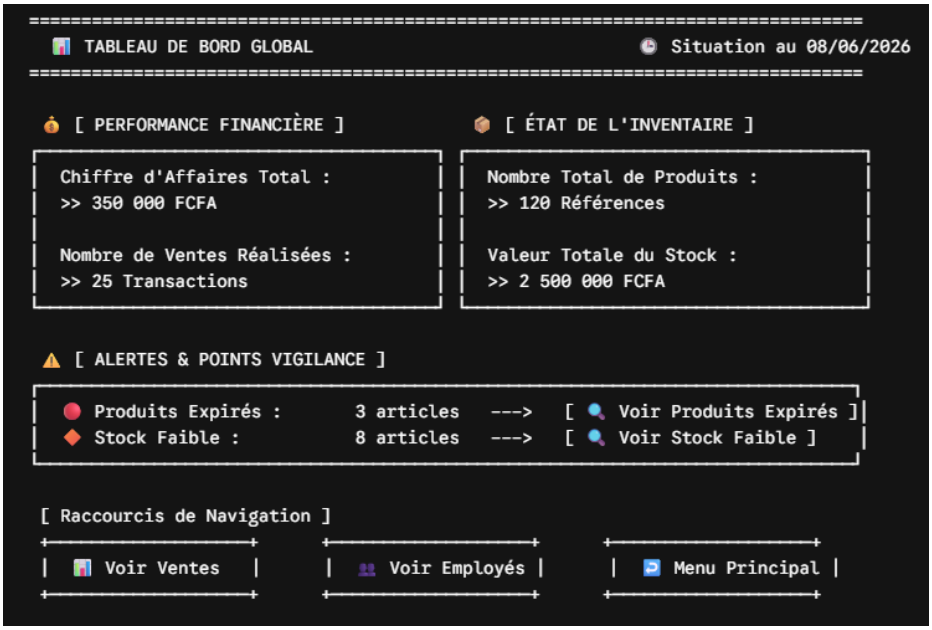
- le chiffre d'affaires total ;
- le nombre de ventes réalisées ;
- les produits les plus vendus ;
- les produits en stock faible ;
- les produits périmés ;
- la valeur totale du stock ;
- la liste des employés.

Créer la classe :
Supermarche
Cette classe représente l'ensemble du supermarché.
Attributs :
private String nom;
private ArrayList<Produit> produits;
private ArrayList<Client> clients;
private ArrayList<Employe> employes;
private ArrayList<Rayon> rayons;
private ArrayList<Vente> ventes;

Méthodes :
ajouterProduit()
ajouterClient()
ajouterEmploye()
ajouterRayon()
enregistrerVente()
calculerChiffreAffaires()
afficherStockFaible()
afficherProduitsExpires()

Interface graphique Dashboard

Créer :
Dashboard.java
Cette interface est réservée à la Direction.
Elle devra afficher :



Le tableau de bord devra être alimenté par les ventes et les stocks réellement enregistrés dans l'application.

EXERCICE 11 : GESTION DES UTILISATEURS ET ACCÈS AUX MODULES

Description métier

Tous les employés ne doivent pas avoir accès aux mêmes fonctionnalités.
Par exemple :

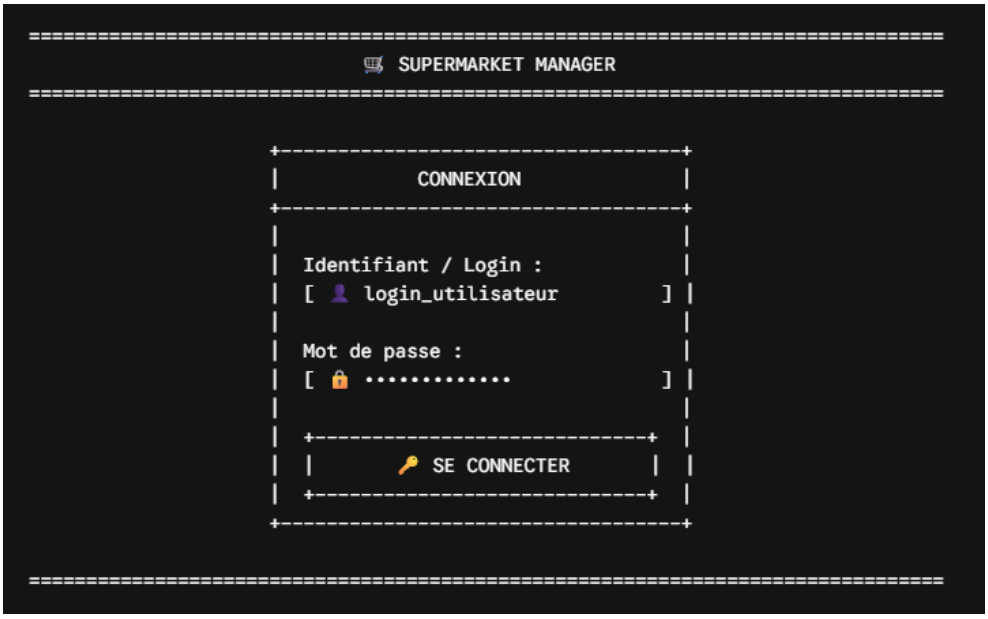
- un caissier doit accéder à la caisse ;
- un magasinier doit accéder au stock ;
- un chef de rayon doit accéder aux rayons ;
- un directeur doit accéder aux statistiques ;
- un comptable doit accéder aux ventes et paiements.

L'application devra donc prévoir une gestion simple des accès.

Créer la classe :
Utilisateur
Attributs :
private String login;
private String motDePasse;
private Employe employe;
private String role;

Méthodes :
seConnecter()
verifierRole()

Interface de connexion
Créer :
FenetreConnexion.java
Au lancement de l'application, l'utilisateur doit d'abord se connecter.
Exemple :



EXERCICE 12 : CLASSE PRINCIPALE DE LANCEMENT

Créer une classe :

Application.java

Cette classe contiendra la méthode :

public static void main(String[] args)

Au démarrage, elle doit :

1. créer quelques produits de test ;
2. créer quelques employés ;
3. créer quelques clients ;
4. ouvrir la fenêtre de connexion ;
5. permettre de tester les modules.

EXERCICE 13 : TRAVAIL UML À FOURNIR

Avant de coder, chaque étudiant devra produire une modélisation UML.

Le rapport devra contenir :

1. Diagramme de cas d'utilisation

Il devra montrer les interactions entre :

- Directeur ;
- Caissier ;
- Magasinier ;
- Chef de rayon ;
- Gestionnaire de stock ;
- Client ;
- Comptable.

Fonctionnalités :

- gérer produits ;
- gérer stock ;
- gérer vente ;
- gérer paiement ;
- imprimer facture ;
- consulter statistiques ;
- gérer employés.

2. Diagramme de classes

Il devra représenter au minimum :

- Produit ;
- ProduitFrais ;
- ProduitElectronique ;
- ProduitArtisanal ;
- Fournisseur ;
- CommandeFournisseur ;
- Stock ;
- Rayon ;
- Employe ;
- Caissier ;
- Magasinier ;
- ChefRayon ;
- Directeur ;
- Client ;
- Vente ;
- Paiement ;
- Facture ;
- Supermarche ;
- Utilisateur.

3. Diagramme de séquence

Faire au moins le diagramme de séquence du scénario :
Réaliser une vente en caisse
Le scénario doit montrer :
Caissier → FenetreCaisse → Vente → Produit → Paiement → Facture

1. **Processus d'approvisionnement des marchandises**

Avant qu'un produit soit vendu aux clients, il doit d'abord être acheté auprès d'un fournisseur.
Lorsqu'un stock devient insuffisant :

- le gestionnaire de stock identifie les produits concernés ;
- une commande fournisseur est créée ;
- le fournisseur prépare et livre les produits ;
- le magasinier réceptionne les marchandises ;
- les quantités reçues sont ajoutées au stock disponible.

L'application devra permettre de conserver l'historique de tous ces mouvements.

Instruction : bien lire les instructions et les respecter scrupuleusement

Le projet est à réaliser par chaque groupe.
Il est demandé à chaque groupe de fournir :

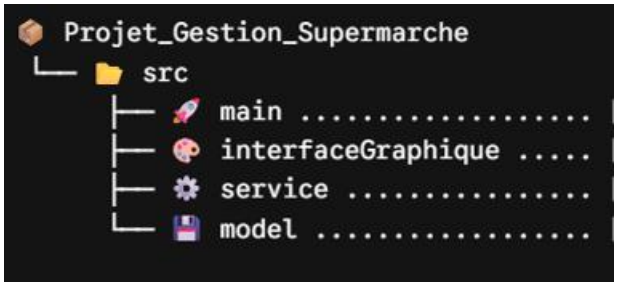
Chaque étudiant devra fournir :

1. Projet Java

Nom du projet :

Projet_Gestion_Supermarche

Structure minimale :



2. Rapport Word

Nom du fichier :

Rapport_Gestion_Supermarche_Numéro_du_Groupe.docx
Le rapport devra contenir :

- page de garde ;
- introduction ;
- description du projet ;
- diagrammes UML ;
- captures d'écran des interfaces ;
- captures d'écran du code ;
- conclusion.

3. Dossier compressé

Nom du dossier : TPPOOJAVA_SUPERMARCHE_NUMERO_GROUPE

Il devra contenir :

Projet_Gestion_Supermarche
Rapport_Gestion_Supermarche_ Numéro_du_Groupe.docx

Compresser le dossier **TPPOOJAVA_SUPERMARCHE_NUMERO_GROUPE** et le **déposer** au plus tard le **08 juillet 2026 à 23H59** dans le répertoire portant le nom du groupe. Le dépôt n'est faisable que par le chef de groupe et le lien pour accéder à ce répertoire lui sera communiqué dans sa boîte mail.

Tout projet déposer le 09 juillet 2026 à partir de 00H00 portera la note 0.